

Lab03-CompileLinux

目标

1. 下载最新的Linux内核
2. 配置下载的内核
3. 编译配置好的内核
4. 重启检查内核是否启用成功

准备

- 安装相关的软件包

代码块

```
1  sudo apt-get update
2  sudo apt-get upgrade
3
4  sudo apt-get install build-essential
5
6  # 内核编译所需的一些依赖模块
7  sudo apt-get install flex bison dwarves
8  sudo apt-get install libssl-dev
9  sudo apt-get install libelf-dev
10 sudo apt-get install libncurses-dev
11
12 # gtk+图形界面相关
13 sudo apt-get install pkg-config
14 sudo apt-get install libgtk2.0-dev
15 sudo apt-get install libcanberra-gtk-module
16 sudo apt-get install glade
17 sudo apt-get install libglade2-dev
```

- 查看当前的内核版本

代码块

```
1  $ uname -r
```

- 检查可用的硬盘空间，确保/分区下有足够的空间

代码块

```
1 $ df -BG
```

```
$ df -BG
```

文件系统	1G的块	已用	可用	已用%	挂载点
tmpfs	1G	1G	1G	1%	/run
/dev/sda3	59G	29G	28G	52%	/
tmpfs	4G	0G	4G	0%	/dev/shm
tmpfs	1G	1G	1G	1%	/run/lock
tmpfs	4G	0G	4G	0%	/run/qemu
/dev/sda2	1G	1G	1G	2%	/boot/efi
tmpfs	1G	1G	1G	1%	/run/user/1000

很有可能没有磁盘空间，建议预留30GB空闲空间

扩展磁盘空间

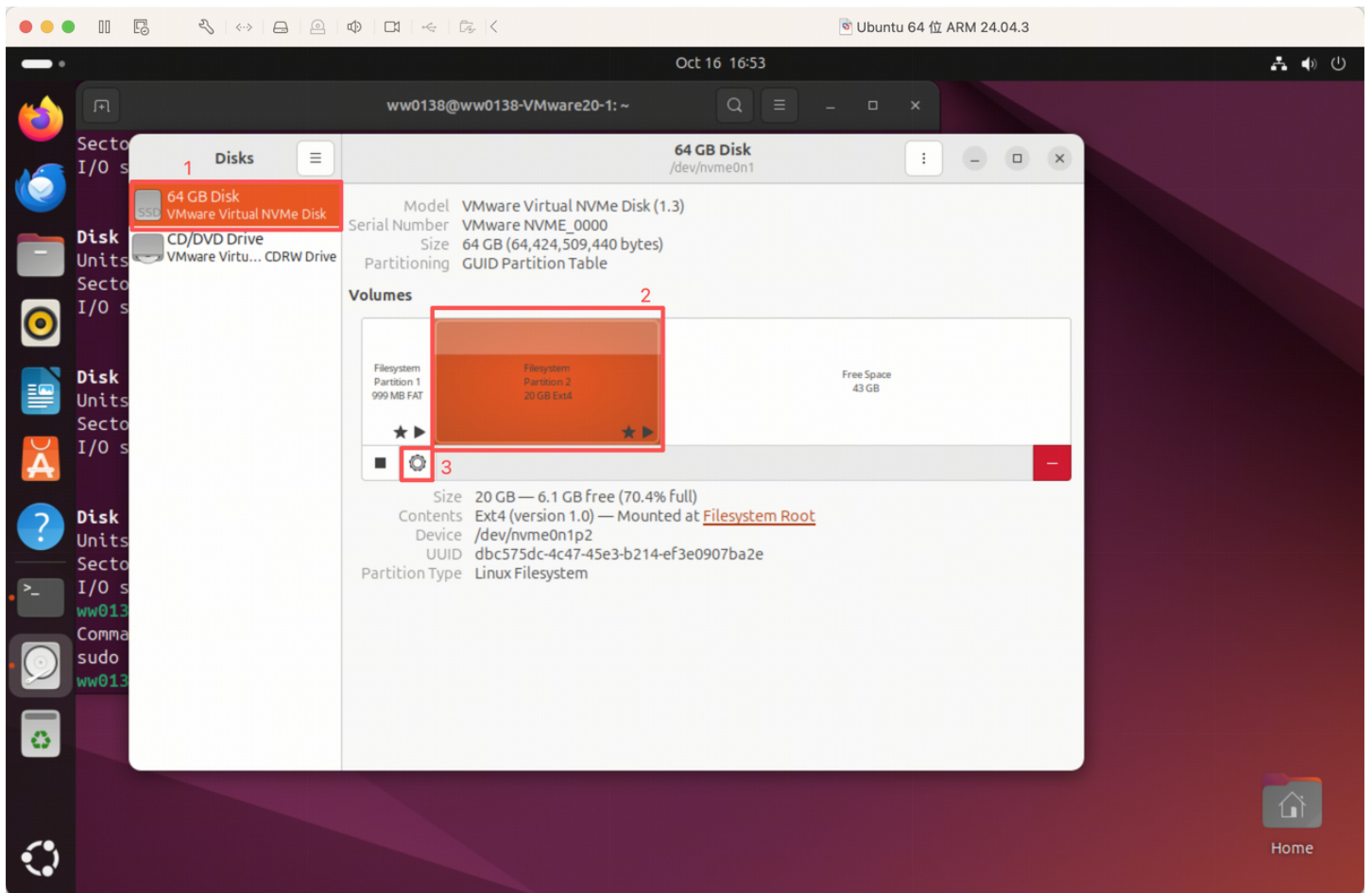
步骤一：VMware中扩展虚拟机的磁盘空间；

选中虚拟机，再选中设置，再选中磁盘，增加磁盘容量，建设置到50~60GB；

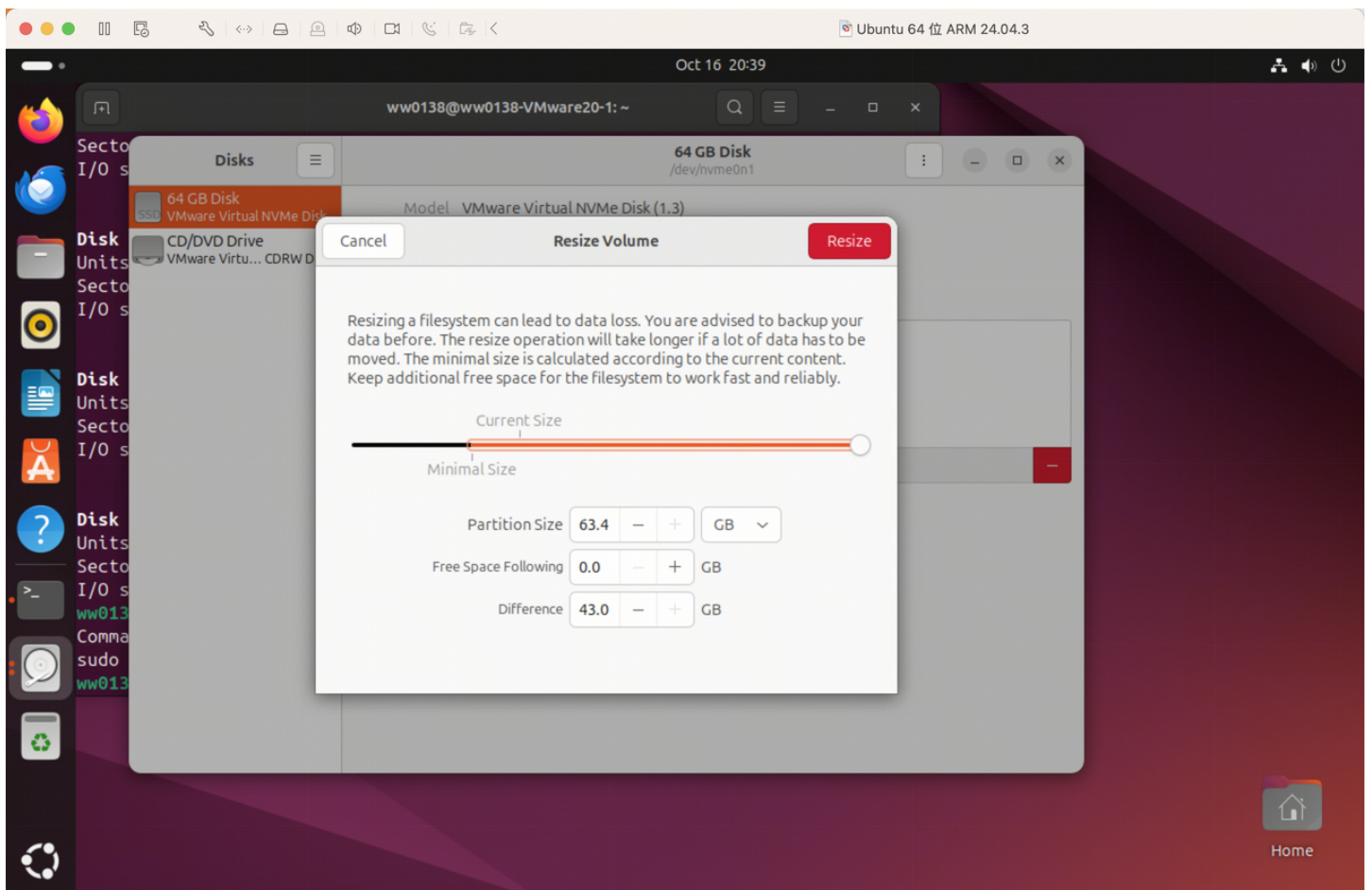
步骤二：使用Ubuntu中的磁盘管理工具Disks调整根分区大小；

应用中心找到Disks启动；

选中磁盘，选中根分区，再点击设置：



选择resize，然后拖动鼠标到最大：



参考文档：https://blog.csdn.net/qq_34160841/article/details/113058756

配置工具

1. `make oldconfig`

- **作用：**基于已有的 `.config` 文件（或指定的配置文件），自动处理新内核版本中新增的配置选项。
- **适用场景：**当你从旧版本内核源码升级到新版本（例如从 5.15 升级到 6.5），且希望保留原有的 `.config` 配置时使用。
- **工作方式：**
 - 新版本内核可能会增加一些旧版本没有的配置项（如新增的驱动、功能开关等）。
 - 运行 `make oldconfig` 后，它会读取现有 `.config` 文件，对于新增的选项，会逐个询问用户选择（默认值通常为内核推荐的选项），用户只需按回车（接受默认）或输入 `y/n/m`（分别表示编译进内核、不编译、编译为模块）即可。
 - 最终生成适配新版本内核的 `.config` 文件，保留了原有配置，同时处理了新增选项。

2. `make gconfig`

- **作用：**以 **图形化界面（GTK+ 窗口）** 展示内核配置选项，方便用户可视化地修改配置。
- **适用场景：**适合习惯图形界面操作的用户，通过窗口、菜单、复选框等直观地配置内核功能（如驱动、文件系统、安全选项等）。
- **依赖：**需要系统安装 GTK+ 开发库（如 `libgtk2.0-dev`），否则会报错。如果缺少依赖，可通过以下命令安装：
- `bash`

代码块

```
1  sudo apt install libgtk2.0-dev
```

- **特点：**界面类似图形化工具，可通过搜索快速定位配置项，比文本界面（如 `menuconfig`）更直观，但需要图形环境支持（如桌面版 Ubuntu，服务器版纯命令行环境无法使用）。

编译及安装中常见错误

- 错误1：

```
In file included from scripts/gendwarfksyms/gendwarfksyms.c:12:
scripts/gendwarfksyms/gendwarfksyms.h:6:10: fatal error: dwarf.h: No such file or directory
   6 | #include <dwarf.h>
     |           ^~~~~~
compilation terminated.
```

查看是否存在相应头文件，如果没有则安装

代码块

```
1 find /usr/include /usr/local/include -name "dwarf.h"
2
3 # 如没有则安装
4 sudo apt install libdwarf-dev
```

如果 `scripts/gendwarfsyms/gendwarfsyms.h` 中直接用 `#include <dwarf.h>` 引用, 但实际 `dwarf.h` 在子目录 (如 `libdwarf/dwarf.h`) , 可手动修改该头文件:

代码块

```
1 # 编辑报错的文件
2 nano scripts/gendwarfsyms/gendwarfsyms.h
```

将其中的:

代码块

```
1 #include <dwarf.h>
```

改为实际路径 (例如) :

代码块

```
1 #include <libdwarf/dwarf.h>
```

保存后重新编译。

- 错误2:

```
In file included from scripts/gendwarfsyms/gendwarfsyms.c:12:
scripts/gendwarfsyms/gendwarfsyms.h:7:10: fatal error: elfutils/libdw.h: No such file or directory
  7 | #include <elfutils/libdw.h>
    |           ^~~~~~
compilation terminated.
```

安装相应包

代码块

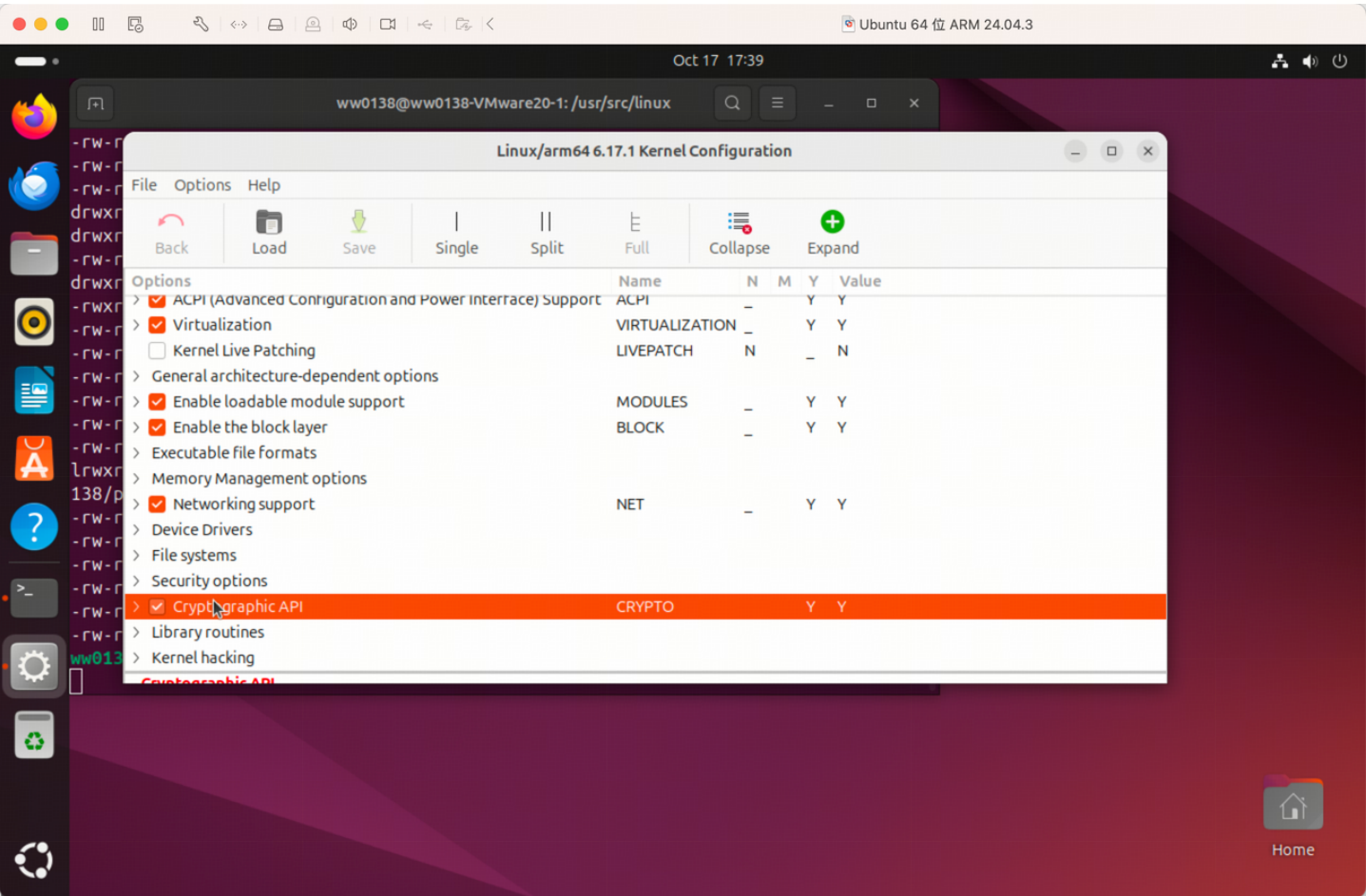
```
1 sudo apt install libdw-dev
```

- 错误3:

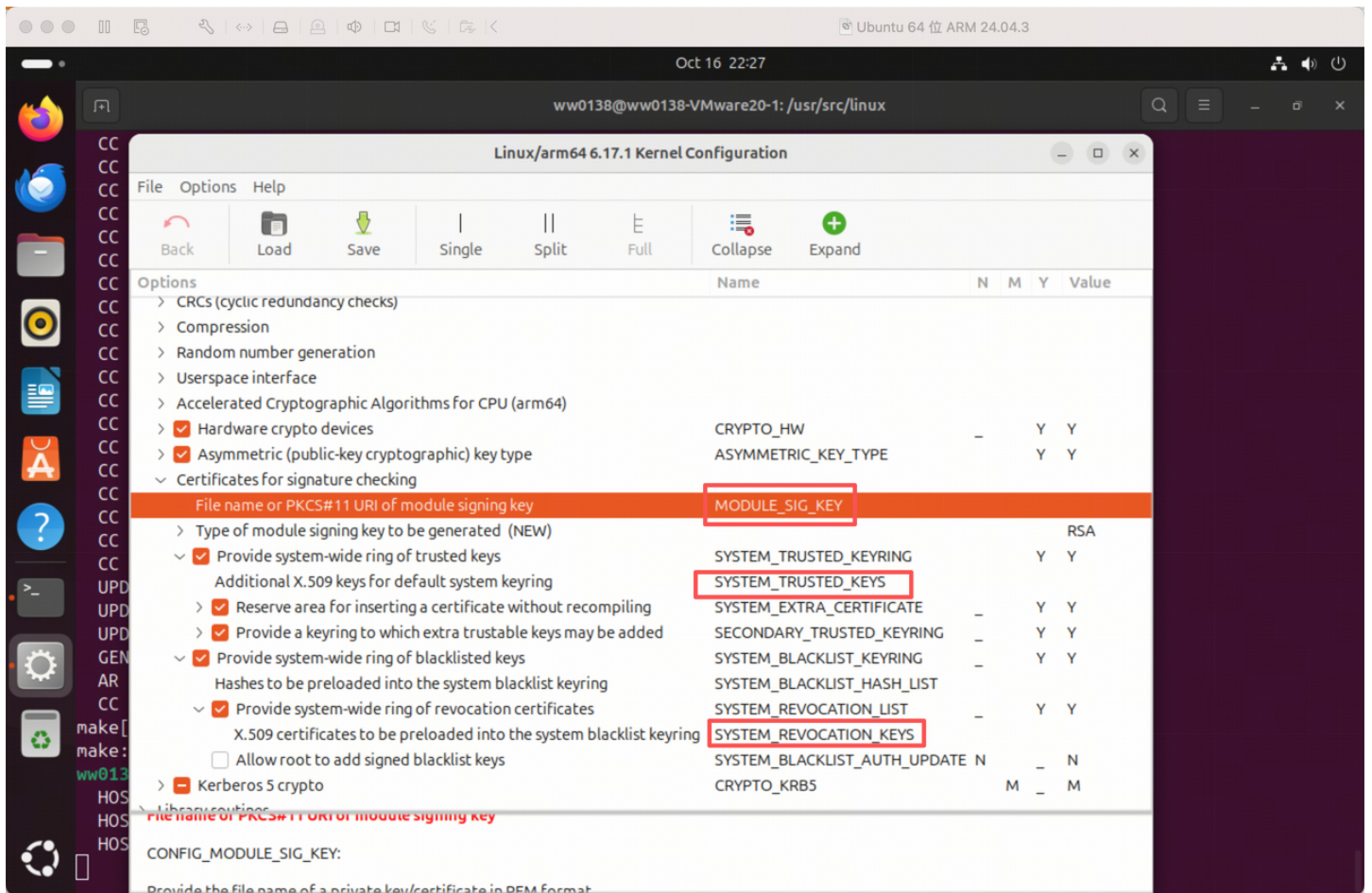
No rule to make target 'debian/canonical-certs.pem', needed by 'certs/x509_certificate_list'.
Stop.

通过内核配置禁用证书验证，需将下面三处设置为空

先下划找到下面的选项



再将三个选项设置为空



• 错误4:

gawk: not found

安装相应包

代码块

```
1 sudo apt install gawk
```

• 错误5:

At main.c:174:

SSL error:1E08010C:DECODER routines::unsupported:

../crypto/encode_decode/decoder_lib.c:101

禁用模块验证

编辑内核配置文件（.config），注释或修改以下行：

代码块

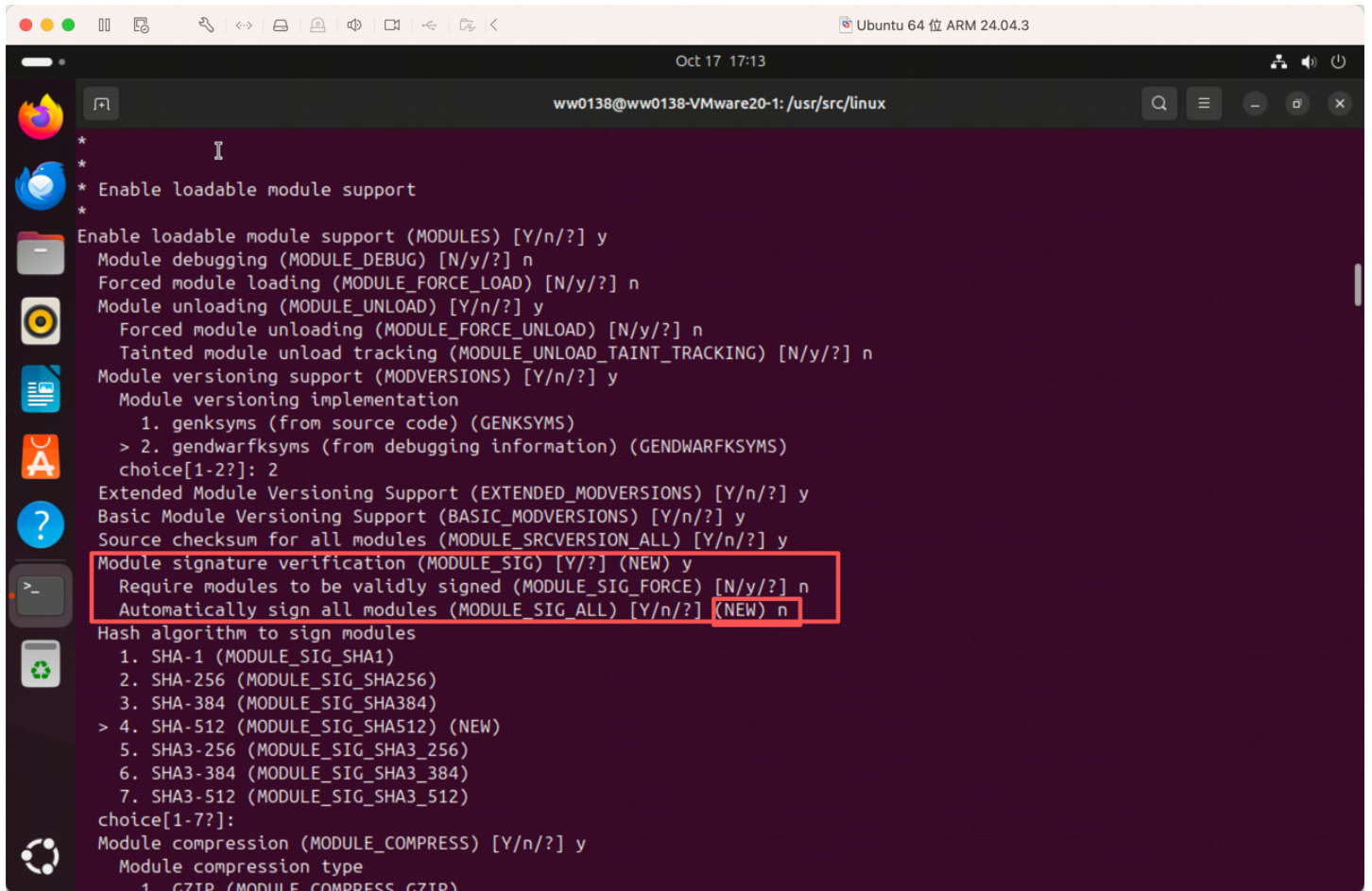
```
1 # CONFIG_MODULE_SIG=y
2 # CONFIG_MODULE_SIG_ALL=y
3 # CONFIG_MODULE_SIG_SHA512=y # 或其他哈希算法选项
4 # CONFIG_MODULE_SIG_KEY="certs/signing_key.pem" # 注释掉密钥路径
```

重新编译

代码块

```
1 make -j$(nproc)
```

会弹出配置窗口，只需在第一次【是否自动签名所有模块】时输入:n，其它都按回车就行；



```
*
* Certificates for signature checking
*
File name or PKCS#11 URI of module signing key (MODULE_SIG_KEY) [certs/signing_key.pem] (NEW)
```

实验流程

1. 下载最新的Linux内核

代码块

```
1 # 下载
2 wget https://cdn.kernel.org/pub/linux/kernel/v6.x/linux-6.17.1.tar.xz
3
4 # 解压
5 tar xvfj linux-6.17.1.tar.xz
```



```
6
7  # 新建软链接
8  sudo ln -s `pwd` /linux-6.17.1 /usr/src/linux
```

2. 配置下载的内核

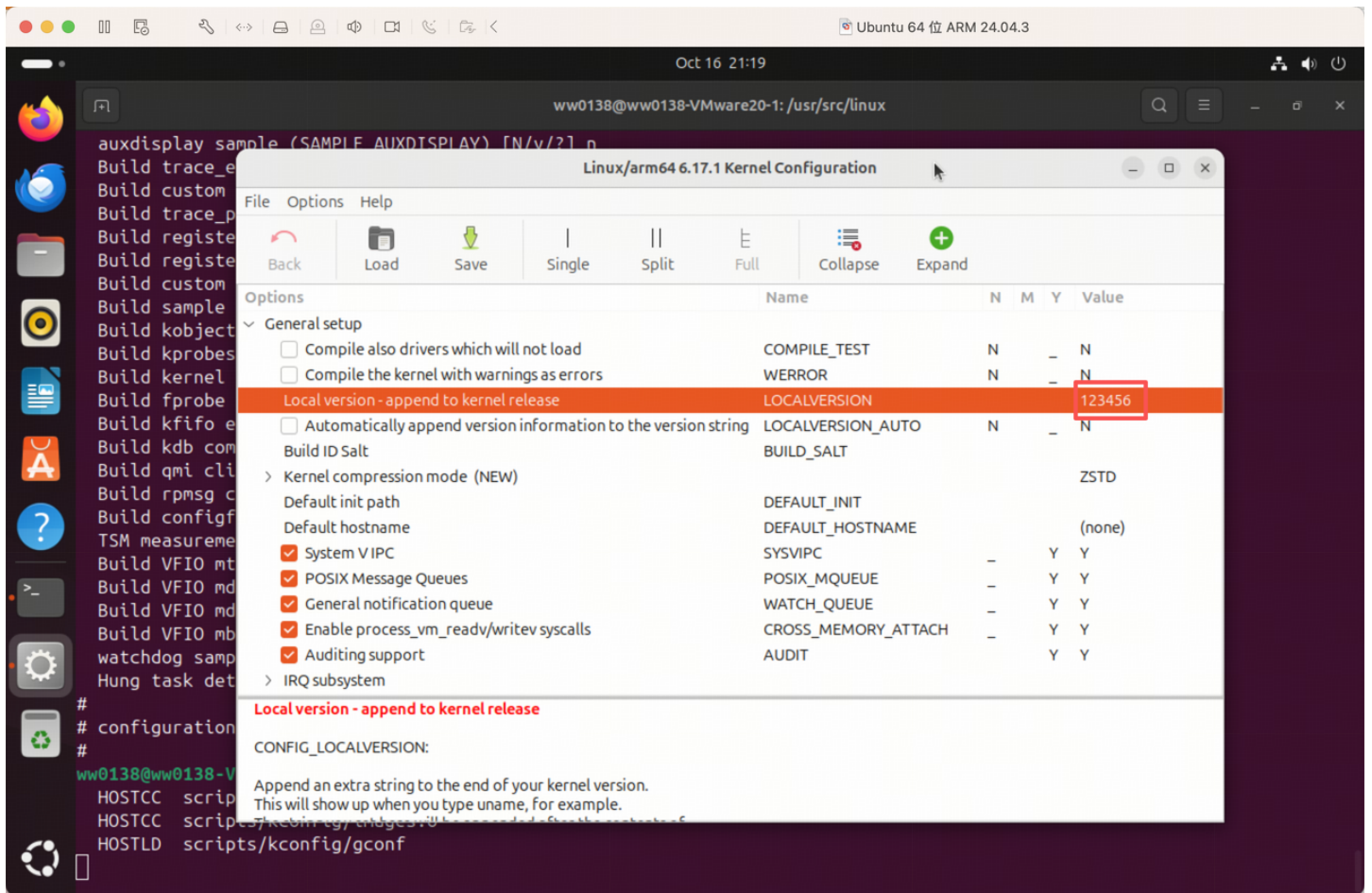
代码块

```
1  # 进入新内核目录
2  cd /usr/src/linux
3  # 复制当前系统的内核配置
4  cp /boot/config-`uname -r` .config
5
6  # 基于已有的.config 文件，自动处理新内核版本中新增的配置选项
7  make oldconfig
8  # 后面一直按回车选择内核推荐的默认配置即可
9
10 # 使用图形化界面展示内核配置选项
11 make gconfig
```

3. 定制配置

添加本地版本号，使用你的学号作为本地版本号！

选中[General setup]，选中[Local version]，将value写成自己的学号，最后点击上方的[Save]



因为内核编译一次时间较长，建议大家再配置后学号后，把常见错误里所需的命令也都先执行一遍

4. 清除之前的编译文件（可选）

如果你想重新编译，执行下面的命令清除之前的缓存文件

代码块

```
1 make clean
```

5. 编译配置好的内核

使用多线程加速编译：

代码块

```
1 make -j$(nproc) 2>&1 | tee log
```

命令解释： `-jN`，N 为 CPU 核心数； `2>&1 | tee log` 将输出也保留在log文件中一份，方便我们查看编译过各中的错误。

此步骤需要执行很长时间，也有可能会出错。为了确认确实正常编译成功了，可再执行一遍make，这次会很快完成。

编译成功后，安装编译好的内核：

代码块

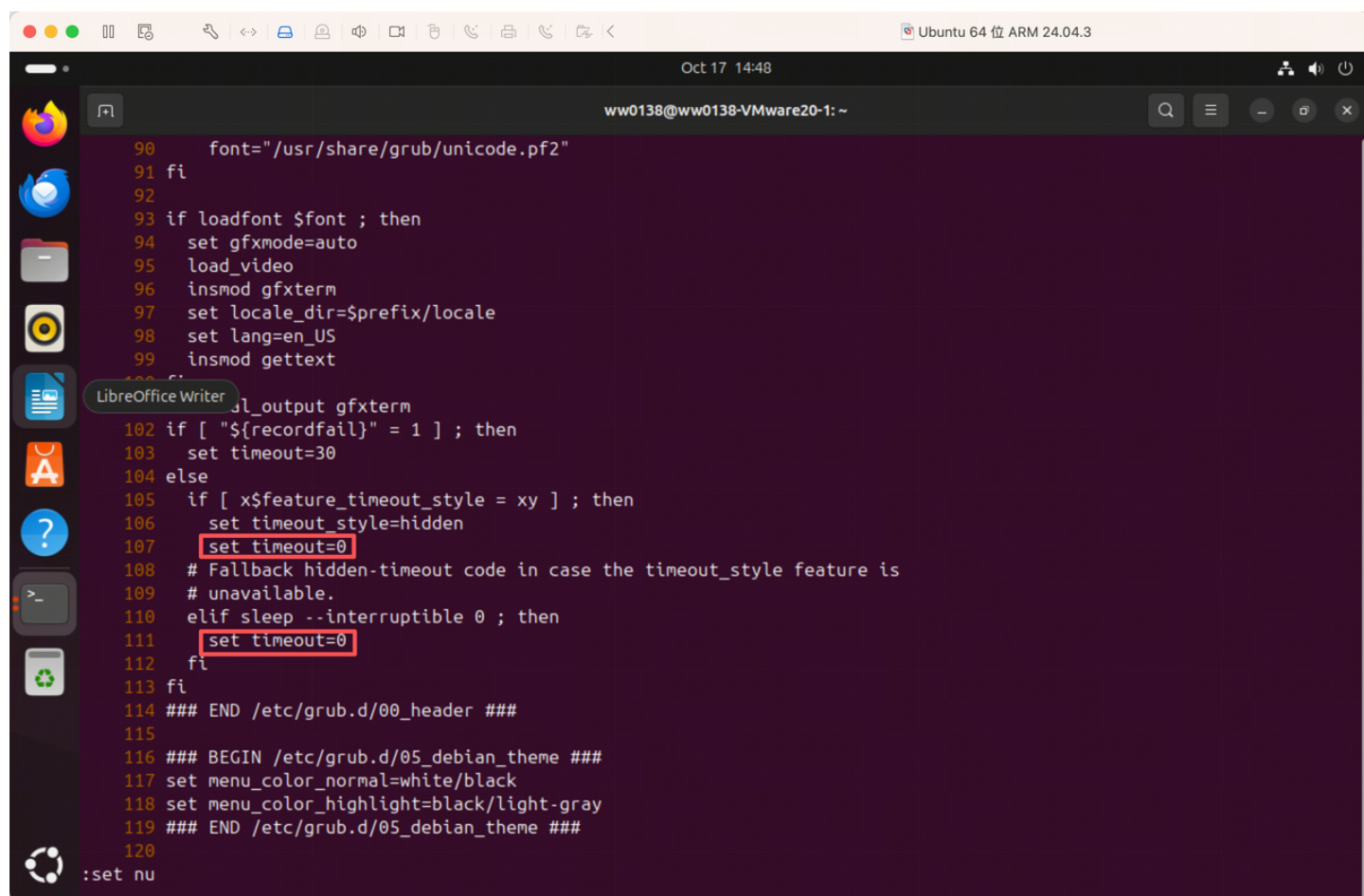
```
1 sudo make modules_install -j$(nproc)
2 sudo make install
```

6. 修改grub.cfg

代码块

```
1 sudo vi /boot/grub/grub.cfg
```

将timeout设为30



```
90 font="/usr/share/grub/unicode.pf2"
91 fi
92
93 if loadfont $font ; then
94   set gfxmode=auto
95   load_video
96   insmod gfxterm
97   set locale_dir=$prefix/locale
98   set lang=en_US
99   insmod gettext
100 fi
101 set default=0
102 if [ "${recordfail}" = 1 ] ; then
103   set timeout=30
104 else
105   if [ x$feature_timeout_style = xy ] ; then
106     set timeout_style=hidden
107     set timeout=0
108   # Fallback hidden-timout code in case the timeout_style feature is
109   # unavailable.
110   elif sleep --interruptible 0 ; then
111     set timeout=0
112   fi
113 fi
114 ### END /etc/grub.d/00_header ###
115
116 ### BEGIN /etc/grub.d/05_debian_theme ###
117 set menu_color_normal=white/black
118 set menu_color_highlight=black/light-gray
119 ### END /etc/grub.d/05_debian_theme ###
120
:set nu
```

7. 重启并检查内核版本

代码块

```
1 sudo reboot
2
3 # 重启进入系统后再次检查内核版本
4 uname -r
```

8. 根据实验流程撰写实验报告，报告请包含重要步骤的截图